OOP CONCEPTS REVISION GUIDE

A Comprehensive Guide to Object-Oriented Principles, Dynamic Dispatch, Diamond MRO & SOLID Architecture

A Concise, Visual, and Mechanism-Focused Interview & Revision Handbook

METADATA FIELD	SPECIFICATION
Subject:	Object-Oriented Programming & Clean Architecture
Target Audience:	SDE / Technical Interviews (Top Tech, FAANG, Product Firms)
Methodology:	Internal Mechanics • State Transitions • Verified Code Recipes • Trade-offs
Source File Consolidated:	Interview Concepts/OOPS.py (534 lines covering OOP pillars, dispatch & exceptions)
Subject Coverage:	Classes/Objects, 4 Pillars, Static vs Dynamic Dispatch, Duck Typing, Diamond MRO, SOLID, Exceptions
Key Architectural Focus:	Encapsulation vs Abstraction, Composition vs Aggregation, C3 Linearization & Failure Isolation
Target Evaluation:	Technical Interview Placement Preparation - Core Engineering Track

CH	CHAPTER TITLE & CORE MECHANISM	CRITICAL INTERVIEW CONCEPTS & CODE PATTERNS
1	Foundations: Classes, Objects, State & Behavior	Object memory layout, State vs Behavior, Object-Based vs Object-Oriented
2	The 4 Pillars of OOP	Encapsulation, Abstraction, Inheritance, Polymorphism; Definitions & Boundaries
3	Encapsulation vs Abstraction Deep Dive	Data Hiding, Private `__x` vs Protected `_x`, Getters/Setters, Public Interfaces
4	Polymorphism: Overloading vs Overriding	Compile-Time (Early Binding) vs Run-Time (Late Binding / Dynamic Dispatch / Duck Typing)
5	Inheritance, Diamond Problem & Python MRO	Single, Multilevel, Multiple; C3 Linearization algorithm (`Class.mro()`), `super()` chaining
6	Abstract Classes vs Interfaces	`abc.ABC` & `@abstractmethod`, Contract enforcement, When to choose each
7	Object Relationships: Association, Aggregation, Composition	IS-A vs HAS-A; Weak ownership vs Strong lifecycle ownership
8	SOLID Design Principles for Clean Architecture	Single Responsibility, Open/Closed, Liskov, Interface Segregation, Dependency Inversion
9	Exception Handling Hierarchy & Best Practices	BaseException hierarchy, try-except-else-finally, Custom Exceptions, Raising vs Handling

CHAPTER 1 - FOUNDATIONS: CLASSES, OBJECTS, STATE & BEHAVIOR

Object-Oriented Programming (OOP) is a programming paradigm organized around discrete entities called **Objects**, which combine data (**State**) and operations (**Behavior**) into unified memory containers.

1.1 Class vs Object

• **Class:** The blueprint, schema, or user-defined type that defines what attributes and methods exist.
• **Object (Instance):** A concrete entity allocated in memory (heap) created according to the class blueprint.
• **Object-Based vs Object-Oriented:** An *Object-Based* language provides objects and methods but lacks inheritance and dynamic polymorphism (e.g., standard JavaScript before ES6). An *Object-Oriented* language supports all 4 pillars including true inheritance and runtime polymorphism.

1.2 Constructors & Memory Instantiation

Code: Python Class Initialization

```
class Account:
    def __init__(self, owner: str, balance: float = 0.0):
        # self is the explicit memory reference to the newly allocated instance
        self.owner = owner          # Instance attribute (State)
        self.balance = balance      # Instance attribute (State)

    def deposit(self, amount: float) -> None:  # Method (Behavior)
        if amount > 0:
            self.balance += amount
```

CHAPTER 2 - THE 4 PILLARS OF OOP

Pillar	Core Technical Definition	Problem It Solves	Real-World Code Concept
1. Encapsulation	Bundling data and methods into a single unit while restricting direct external access to internal state.	Protects object invariants from uncontrolled external mutation.	Private variables (`__balance`), getters/setters, properties.
2. Abstraction	Hiding internal implementation complexity and exposing only a clean, essential public interface.	Reduces cognitive load; decoupling callers from implementation changes.	Abstract Base Classes (`abc.ABC`), interfaces, API boundaries.
3. Inheritance	Mechanism where a new class derives properties and behaviors from an existing parent class.	Eliminates duplicate boilerplate code; establishes IS-A hierarchies.	`class Dog(Animal):`, `super().__init__()`.
4. Polymorphism	Ability of different classes to respond to the same interface or method call with their own distinct behaviors.	Enables open/closed extensible code without giant switch/if-else ladders.	Method Overriding, Dynamic Dispatch, Duck Typing.

CHAPTER 3 - ENCAPSULATION VS ABSTRACTION DEEP DIVE

3.1 The Crucial Difference

Although frequently confused in technical interviews, Encapsulation and Abstraction operate on different axes:

• **Encapsulation is about IMPLEMENTATION / CONTAINMENT:** *'How does the object protect its data?'* It wraps data inside boundaries and guards mutations via methods.

• **Abstraction is about DESIGN / INTERFACE:** *'What does the object expose to the outside world?'* It hides internal mechanics so the consumer only cares about what it does, not how it does it.

3.2 Python Access Modifiers & Name Mangling

Modifier Type	Syntax in Python	Visibility / Enforcement Rule	Internal Runtime Mechanism
Public	self.name	Accessible from anywhere (inside and outside class).	Direct attribute lookup in obj.__dict__.
Protected	self._name	Convention: meant for internal class and subclass use only.	NOT enforced by interpreter; treated as public internally.
Private	self.__name	Restricted: cannot be accessed directly by name from outside.	**Name Mangling:** Python renames attribute to _ClassName__name!

Code: Encapsulation with Managed Properties

```
class BankAccount:
    def __init__(self, initial_balance: float):
        self.__balance = initial_balance  # Private (name-mangled to _BankAccount__balance)

    @property
    def balance(self) -> float:           # Clean getter
        return self.__balance

    def withdraw(self, amount: float) -> bool:  # Guarded mutation (Encapsulation)
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            return True
        return False
```

CHAPTER 4 - POLYMORPHISM: OVERLOADING VS OVERRIDING

Dimension	Compile-Time Polymorphism (Overloading)	Run-Time Polymorphism (Overriding)
Binding Time	**Early / Static Binding:** Compiler selects method at compile time.	**Late / Dynamic Binding:** Runtime inspects object type to dispatch method.
Method Signature	Same method name, **different parameter types or counts**.	Same method name, **exact identical signature & parameters**.
Class Context	Occurs within the **same class**.	Occurs across a **Parent-Child inheritance hierarchy**.
Python Support	Not supported natively (later definition overwrites earlier). Simulated via default args (`x=None`) or `functools.singledispatch`.	Fully supported natively via virtual method tables and dynamic attribute resolution.

4.1 Duck Typing in Python

'If it walks like a duck and quacks like a duck, it's a duck.'
In Python, polymorphism does not require inheriting from a common base class. Any object that implements the required method signature can be passed to a consumer function.

Code: Duck Typing Dynamic Dispatch

```
class PDFParser:
    def parse(self, stream): return "Parsing PDF"

class JSONParser:
    def parse(self, stream): return "Parsing JSON"

def process_file(parser, stream):
    # Dynamic Dispatch: executes parser's parse() regardless of class hierarchy
    return parser.parse(stream)
```

CHAPTER 5 - INHERITANCE, DIAMOND PROBLEM & PYTHON MRO

5.1 The Diamond Problem in Multiple Inheritance

When Class D inherits from both Class B and Class C, which both inherit from Class A: If both B and C override a method from A, which version should D inherit? Languages like Java ban multiple class inheritance to avoid ambiguity. Python resolves this via **C3 Linearization (Method Resolution Order / MRO)**.

5.2 C3 Linearization Algorithm & super() Chaining

Python's MRO enforces three mathematical guarantees:
1. Children precede their parents.
2. Multiple parents are checked in the order listed in the class declaration.
3. Monotonicity: if A precedes B in one class's MRO, A precedes B in all derived classes' MROs.

Code: Python C3 MRO Execution

```
class A:
    def action(self): return "A"

class B(A):
    def action(self): return "B -> " + super().action()

class C(A):
    def action(self): return "C -> " + super().action()

class D(B, C):
    def action(self): return "D -> " + super().action()

# Inspecting D's MRO:
# D.mro() == [D, B, C, A, object]
d = D()
print(d.action())  # Output: "D -> B -> C -> A" (Cooperative super() chaining!)
```

CHAPTER 6 - ABSTRACT CLASSES VS INTERFACES

Dimension	Abstract Class (`abc.ABC`)	Interface (Pure Protocol / Java Interface)
Method Implementation	Can contain **both** abstract methods AND concrete implemented methods.	Contains **only** method signatures (no concrete method bodies).
State / Attributes	Can define instance variables and stateful constructors.	Cannot store instance state; only constants.
Inheritance Model	Single class inheritance in most languages.	A class can implement multiple independent interfaces.
Core Architectural Purpose	Base for a family of closely related objects sharing common code.	Defines a behavioral contract for unrelated classes across systems.

Code: Abstract Base Class Contract

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def charge(self, amount: float) -> bool:
        pass  # Subclasses MUST implement this method!

    def log_transaction(self, tx_id: str):  # Concrete shared method
        print(f"Transaction logged: {tx_id}")

class StripeGateway(PaymentGateway):
    def charge(self, amount: float) -> bool:
        return True  # Valid concrete implementation
```

CHAPTER 7 - OBJECT RELATIONSHIPS: ASSOCIATION, AGGREGATION, COMPOSITION

Relationship	UML Notation	Ownership Strength	Lifecycle Dependency	Code Example
Association	Plain Arrow	None (Peer-to-peer / Uses)	Independent: Objects exist independently of each other.	Doctor and Patient; Teacher and Student.
Aggregation	Hollow Diamond	Weak Ownership (HAS-A)	Loose: If parent is destroyed, child continues to exist.	Department and Professors (deleting Dept does not delete professors).
Composition	Filled Diamond	**Strong Ownership (PART-OF)**	Strict: If parent is destroyed, **children are destroyed with it!**	House and Rooms; Order and OrderLineItems.

Code: Composition Implementation

```
# Composition Example: Strong Lifecycle Ownership
class Room:
    def __init__(self, name): self.name = name

class House:
    def __init__(self):
        # House creates and owns Room instances directly
        self.rooms = [Room("Living"), Room("Kitchen")]

    def __del__(self):
        # When House memory is freed, Room objects are destroyed
        del self.rooms
```

CHAPTER 8 - SOLID DESIGN PRINCIPLES FOR CLEAN ARCHITECTURE

Principle	Formal Name	Core Rule / Invariant	Violation Symptom
S	Single Responsibility (SRP)	A class should have **one, and only one, reason to change**.	God class handling business logic, database queries, and email formatting.
O	Open/Closed (OCP)	Software entities should be **open for extension, but closed for modification**.	Adding a new payment type requires modifying an existing 500-line if-else function.
L	Liskov Substitution (LSP)	Subtypes must be substitutable for their base types without altering program correctness.	Square inherits from Rectangle, but setting width alters height and breaks assertions.
I	Interface Segregation (ISP)	Clients should not be forced to depend on interfaces they do not use.	Fat interface with 50 methods forcing dummy empty implementations.
D	Dependency Inversion (DIP)	High-level modules should depend on abstractions, not concrete low-level details.	OrderService directly instantiates `MySQLDatabase` via `new` instead of receiving a `Database` interface.

CHAPTER 9 - EXCEPTION HANDLING HIERARCHY & BEST PRACTICES

9.1 Python Exception Class Hierarchy

All Python exceptions inherit from BaseException:

- BaseException -> KeyboardInterrupt, SystemExit (Should NEVER be caught in generic handlers!).
- Exception -> Base for all non-system-exiting exceptions (ArithmeticError, LookupError, ValueError).
- **Golden Rule:** Never write a bare except: or catch BaseException; it intercepts Ctrl+C and process termination signals!

9.2 Custom Exceptions & Clean Raising

Code: Custom Exception Pattern

```
# Custom Exception Hierarchy
class ApplicationError(Exception):
    """Base exception for application business errors."""
    pass

class InsufficientFundsError(ApplicationError):
    def __init__(self, balance: float, amount: float):
        super().__init__(f"Attempted to withdraw {amount} with balance {balance}")
        self.balance = balance
        self.amount = amount

# Proper try-except-else-finally pattern
try:
    account.withdraw(500)
except InsufficientFundsError as e:
    logger.warning("Overdraft prevented: %s", e)
except ApplicationError as e:
    logger.error("Application fault: %s", e)
else:
    notify_user_success()  # Runs ONLY if NO exception was raised!
finally:
    db_session.close()     # Runs unconditionally (cleanup)
```